Disk sharing using EBS and EFS

Description

GOAL: Configure an Amazon EBS volume and Amazon EFS system such that storage can be effectively shared between different EC2 instances.

1. Create and set up an EBS volume.
2. Verify the working and data persistence across instances.
3. Study about Elastic File System (EFS) in AWS.
4. Configure an EFS shared filesystem between different EC2 instances simultaneously.

PART 1 — Amazon EBS (Elastic Block Store)

Task 1 — Attach One EBS Volume Between Two EC2 Instances

Goal: Attach a single EBS volume to one EC2 instance, detach it, then attach it to another instance while preserving the data natively.

Prerequisites

- Two EC2 Linux instances (e.g., Instance1 and Instance2).
- Both instances must reside in the exact same Availability Zone (AZ).
- Configured SSH access to both instances.

Step 1 — Create EBS Volume

Open the AWS Management Console, navigate to **EC2 → Volumes**, and click **Create Volume**. Provision the volume with the following configuration details:

Setting	Value
Volume Type	gp3
Size	10 GiB
Availability Zone	Must match the exact AZ of Instance1 & Instance2

Step 2 — Attach Volume to Instance1

Select the newly created volume, go to Actions → Attach Volume. Select Instance1 from the drop-down menu and specify the Device Name as /dev/sdf. Click Attach.

Step 3 — Verify Disk in Instance1

SSH into Instance1 and check available block devices using the command below. The new disk typically maps to an NVMe device link name (e.g., nvme1n1).

```
lsblk
```

Step 4 — Create Filesystem

Run the file system creation command ONLY the first time you set up the volume. Do not format a device that already contains data.

```
mkfs -t xfs /dev/nvme1n1
```

Step 5 — Create Mount Point

Create a clean storage target directory directory on the host environment:

```
mkdir /data
```

Step 6 — Mount Volume

Mount the block device to the data path directory and safely verify the operating system link architecture:

```
mount /dev/nvme1n1 /data
df -h
```

Step 7 — Create Sample File

Write mock context variables onto the filesystem block directly to trace live multi-host validation state transfers:

```
echo "Hello from Instance1" | tee /data/test.txt
cat /data/test.txt
```

Step 8 — Unmount Before Detach

Safely detach the directory mapping node prior to infrastructure isolation tasks to fully minimize risk of filesystem cache corruption updates:

```
umount /data
```

Step 9 — Detach EBS Volume

Navigate back to **EC2 → Volumes**. Choose your volume, select **Actions → Detach Volume**, and wait until the console dashboard indicates the volume state has transitioned back to *available*.

Step 10 — Attach Volume to Instance2

Select the exact same volume block, click **Actions → Attach Volume**, and pick **Instance2**. Keep device name configurations structured to standard device endpoints (e.g., */dev/sdf*).

Step 11 — Mount Volume in Instance2

SSH securely into Instance2. Map the localized directories directly to mount points. IMPORTANT: Do NOT run 'mkfs' again on this instance, as doing so will format the volume and erase your data.

```
mkdir /data
mount /dev/nvme1n1 /data
```

Step 12 — Verify Old Data Exists

Validate that the data created from Instance1 persists smoothly across host handoffs:

```
cat /data/test.txt

# Expected Output:
# Hello from Instance1
```

Conclusion: This successful output proves that EBS block data persists independently after detaching structural storage, and that volumes can be cleanly migrated between instances.

PART 2 — EBS Multi-Attach Architecture & Limits

AWS EBS Multi-Attach enables a single Provisioned IOPS EBS volume to be concurrently attached to multiple EC2 nodes within the same Availability Zone. Each connected instance retains full read and write capabilities over the shared block layer.

Multi-Attach Key Technical Limitations

Limitation Metric	Details & Requirements
Supported Volume Types	io1 and io2 Provisioned IOPS SSDs only.
AZ Alignment Boundary	Strictly constrained to the same Availability Zone (AZ).
Shared Access Integrity Risk	Simultaneous unmanaged block write calls can quickly lead to file corruption.
Preferred System Setup	Requires specialized Cluster File Systems (e.g., GFS2, OCFS2) to coordinate node changes.

Real Production Use Cases for Multi-Attach

- **Clustered Databases:** Enables enterprise deployments like Oracle Real Application Clusters (RAC) where active multiple engine instances must access structural data layers concurrently.
- **Failover Clusters:** Supports high-availability active-passive environments. If a primary active compute instance goes offline, an identical secondary standby node takes control of the block storage without delay.
- **Shared Enterprise Applications:** Architected for applications requiring shared block storage paths, high system availability, and low network latency access bounds.

Why Multi-Attach is Not Ideal for Standard File Sharing

Standard environments do not have a built-in coordination layer for block storage. If standard operating file wrappers write data simultaneously without cluster coordination, metadata blocks overlap, resulting in immediate data corruption and file system inconsistencies. For general file sharing utilities, Amazon EFS is the recommended best practice architecture choice.

PART 3 — Amazon Elastic File System (EFS)

Amazon Elastic File System (EFS) is a fully managed, serverless, elastic shared file storage service. Unlike block storage options, EFS supports shared access across thousands of EC2 instances simultaneously over a standard network layer.

Protocol Framework: EFS leverages the Network File System Version 4 (NFSv4.1/NFSv4) protocol standard, executing requests over target **TCP Port 2049**.

PART 4 — Configure EFS Shared Between Multiple EC2 Instances

Goal: Provision an elastic network file system and mount it onto two EC2 host nodes concurrently to facilitate live bidirectional file syncing.

Prerequisites

- Two functional EC2 Linux instances.
- Deployment within the exact same Virtual Private Cloud (VPC).
- Associated network Security Groups allowing inter-group traffic mappings.

Step 1 — Create EFS File System

Navigate to the EFS console dashboard and click **Create File System**. Apply the default infrastructure parameters below:

Setting Description	Deployment Value
Virtual Private Cloud (VPC)	Select the exact same VPC used by the EC2 instances
Performance Mode	General Purpose
Storage Class Tier	Standard

Step 2 — Configure Security Group Rules

To allow incoming traffic from your instances, locate the Security Group assigned to your EFS mount targets and add an inbound rule:

Type	Protocol	Port Range	Source Boundary

NFS	TCP	2049	EC2 Instances Security Group

Step 3 — Install NFS Utilities

Execute the package installation step across BOTH EC2 instances to verify system compatibility for Network File System client drivers:

```
sudo yum install nfs-utils -y
```

Step 4 — Create Mount Directory

Create a dedicated directory path on both servers to serve as the EFS mount point:

```
sudo mkdir /efs
```

Step 5 — Mount EFS Using IP Address

If DNS resolution is unavailable in your lab sandbox, you can mount EFS directly using the Mount Target IP address. Locate this IP via EFS Console → Network Tab. Example: 172.31.25.120.

```
sudo mount -t nfs4 -o nfsvers=4.1 172.31.25.120:/ /efs
```

Note: Ensure your security groups allow traffic on TCP port 2049. While IP mounting is helpful for troubleshooting, using the EFS DNS name is standard practice for production environments.

Step 6 — Verify Mount

Confirm that the network partition structure reports a clean, valid link execution status:

```
df -h
# OR
mount | grep nfs
```

Step 7 — Test Shared Access (Instance1 → Instance2)

Generate a test file on Instance1 and verify it immediately displays on Instance2:

```
# Run on Instance1
echo "Hello from Instance1" | sudo tee /efs/shared.txt

# Run on Instance2 to verify
cat /efs/shared.txt

# Expected Output:
# Hello from Instance1
```

Step 8 — Reverse Verification (Instance2 → Instance1)

Perform a reverse check by creating a file on Instance2 and validating its visibility from Instance1:

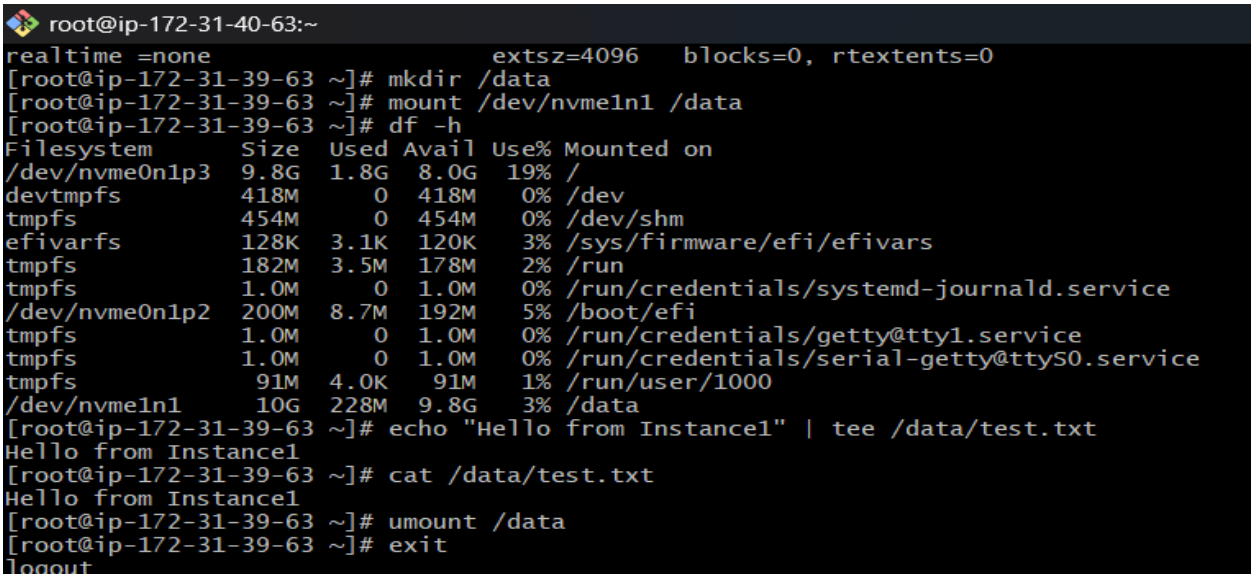
```
# Run on Instance2
echo "Hello from Instance2" | sudo tee /efs/test2.txt

# Run on Instance1 to verify
cat /efs/test2.txt
```

Success: Shared storage setup has been successfully validated. Both instances now have simultaneous, real-time read and write access to the central EFS filesystem.

Output Pictures

EBS



```
Rejin@LAPTOP-I5TH26SA MINGW64 ~/downloads
$ ssh -i "internship-key.pem" ec2-user@ec2-65-0-184-99.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-65-0-184-99.ap-south-1.compute.amazonaws.com (65.0.184.99)' can't be established.
ED25519 key fingerprint is: SHA256:L6KdUornOc9FhWlH+AiiKZZ8W8v688WwTm22DAOrTf8
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-65-0-184-99.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
[ec2-user@ip-172-31-40-63 ~]$ sudo -i
[root@ip-172-31-40-63 ~]# lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1      259:0    0   10G  0 disk
├─nvme0n1p1 259:1    0    1M  0 part
├─nvme0n1p2 259:2    0  200M  0 part /boot/efi
└─nvme0n1p3 259:3    0  9.8G  0 part /
nvme1n1      259:4    0   10G  0 disk
[root@ip-172-31-40-63 ~]# mkdir /data
[root@ip-172-31-40-63 ~]# mount /dev/nvme1n1 /data
[root@ip-172-31-40-63 ~]# cat /data/test.txt
Hello from Instance1
[root@ip-172-31-40-63 ~]#
```

EFS